LAB MANUAL

Error Handling in Python

Introduction

Errors are unavoidable in programming. Even carefully written programs can encounter unexpected situations such as invalid user input, missing files, or division by zero. Python provides a robust mechanism for detecting and managing these situations without crashing the program.

Error handling ensures software reliability by gracefully managing unexpected runtime failures. Instead of allowing a program to stop abruptly, Python lets developers anticipate and respond to problems in a controlled manner.

Understanding error handling is essential for building resilient applications used in software development, data science, artificial intelligence, and automation, where uninterrupted execution is critical.

Learning Objectives

After completing this lab manual, students will be able to:

- Understand the difference between syntax errors and runtime exceptions.
- Use `try` and `except` blocks to handle exceptions.
- Catch specific exceptions for targeted error recovery.
- Handle multiple exceptions in a single program.
- Use the `else` block with exception handling.
- Use the `finally` block for guaranteed execution.

- Trigger exceptions intentionally using `raise`.
- Handle unknown exceptions using the base `Exception` class.
- Apply best practices for writing clean, resilient exception-handling code.

Theory Notes

Syntax Errors vs. Runtime Exceptions

Python programs can fail for two different reasons.

Syntax errors occur when the code is written incorrectly and does not follow Python's grammar rules. These errors are detected before the program runs.

```
print("Hello"
```

Output

```
SyntaxError: unexpected EOF while parsing
```

Runtime exceptions, on the other hand, occur while the program is executing, even though the code is syntactically correct.

```
print(10 / 0)
```

Output

```
ZeroDivisionError: division by zero
```

Understanding this distinction between incorrect code construction and operational execution failures is the foundation of exception handling.

Introduction to Error Handling

Error handling ensures software reliability by gracefully managing unexpected runtime failures. In Python, `try` and `except` blocks are used to catch and handle exceptions, preventing the program from crashing.

Syntax

```
try:
    # code that may cause an error
except:
    # code that runs if an error occurs
```

Example

```
try:
    number = int(input("Enter a number: "))
    print(10 / number)
except:
    print("An error occurred.")
```

Output

```
Enter a number: 0
An error occurred.
```

The program continues running instead of crashing.

Targeted Exception Handling

Catching specific exceptions prevents masking unrelated bugs and allows targeted recovery. Instead of catching all errors generically, Python allows you to specify the exact exception type.

Syntax

```
try:
    # risky code
except ExceptionType:
    # handle specific error
```

Example

```
try:
    number = int(input("Enter a number: "))
    print(10 / number)
except ZeroDivisionError:
    print("You cannot divide by zero.")
except ValueError:
    print("Please enter a valid number.")
```

Output

```
Enter a number: 0

You cannot divide by zero.
```

Targeting specific exceptions makes debugging easier and prevents the program from hiding unrelated issues.

Handling Multiple Exceptions

Python allows handling different exceptions in distinct ways or grouping them.

Handling exceptions separately

```
try:

    value = int("abc")

except ValueError:

    print("Invalid value.")

except TypeError:

    print("Invalid type.")
```

Output

```
Invalid value.
```

Grouping exceptions together

```
try:
    value = int("abc")
except (ValueError, TypeError):
    print("Invalid value or type.")
```

Output

```
Invalid value or type.
```

Grouping exceptions is useful when the same recovery action applies to multiple error types.

The else Block

The `else` block runs only if no exception occurs in the `try` block.

Syntax

```
try:
    # risky code
except ExceptionType:
    # handle error
else:
    # runs if no error occurred
```

Example

```
try:
    number = int(input("Enter a number: "))
except ValueError:
    print("Invalid input.")
else:
    print("You entered:", number)
```

Output

```
Enter a number: 25

You entered: 25
```

The `else` block helps separate the "success path" from the "error path" for cleaner code.

Guaranteed Execution with finally

The `finally` block executes unconditionally to ensure a clean state and resource recovery, regardless of whether an exception occurred.

Syntax

```
try:
    # risky code
except ExceptionType:
    # handle error
finally:
    # always executes
```

Example

```
try:
    file = open("data.txt", "r")
    print(file.read())
except FileNotFoundError:
    print("File not found.")
finally:
    print("Execution finished.")
```

Output

```
File not found.
Execution finished.
```

The `finally` block is commonly used to close files, release resources, or print completion messages.

Triggering Exceptions with raise

Use the `raise` keyword to trigger exceptions when invalid states are detected intentionally.

Syntax

```
raise ExceptionType("Custom error message")
```

Example

```
def check_age(age):
    if age < 0:
        raise ValueError("Age cannot be negative")
    print("Age is valid:", age)
```

```
check_age(-5)
```

Output

```
ValueError: Age cannot be negative
```

The `raise` keyword allows developers to enforce rules and validate data within their own functions.

Handling Unknown Exceptions

When the exact exception type is unknown, use Python's base `Exception` class to capture errors safely.

Example

```
try:
    result = 10 / "two"
except Exception as e:
    print("An error occurred:", e)
```

Output

```
An error occurred: unsupported operand type(s) for /: 'int' and 'str'
```

Using the base `Exception` class serves as a safety net, but it should be used carefully to ensure specific errors are not accidentally hidden.

Best Practices for Exception Handling

Effective exception handling requires precision, minimal scope, and meaningful logging.

- Catch specific exceptions rather than using a bare `except:`.
- Keep the `try` block as small as possible.
- Provide clear, meaningful error messages.
- Avoid silently ignoring exceptions (never use `except: pass` without reason).
- Use `finally` to release resources such as files or network connections.
- Log errors for debugging rather than only printing them in larger applications.

Resilient Exception Handling

The `try-except-else-finally-raise` structure provides a comprehensive framework for building resilient software.

Complete Example

```
try:
    number = int(input("Enter a number: "))
    result = 10 / number
except ZeroDivisionError:
    print("Cannot divide by zero.")
except ValueError:
    print("Invalid number entered.")
else:
```

```
    print("Result:", result)
finally:
    print("Program execution completed.")
```

Output (if input is 0)

```
Enter a number: 0
Cannot divide by zero.
Program execution completed.
```

This structure ensures that valid results are processed, errors are handled gracefully, and cleanup always occurs.

Syntax Reference Sheet

Basic try-except

```
try:
    pass
```

```
except:
    pass
```

Specific exception

```
try:
    pass
except ExceptionType:
    pass
```

Multiple exceptions

```
try:
    pass
except ExceptionType1:
    pass
except ExceptionType2:
    pass
```

Grouped exceptions

```
try:
    pass
except (ExceptionType1, ExceptionType2):
    pass
```

try-except-else

```
try:
    pass
except ExceptionType:
    pass
else:
    pass
```

try-except-finally

```
try:
    pass
except ExceptionType:
    pass
finally:
    pass
```

Raising an exception

```
raise ExceptionType("message")
```

Catching unknown exceptions

```
try:
    pass
except Exception as e:
    pass
```

Worked Examples

Example 1: Division with Error Handling

```
def divide(a, b):
    try:
        return a / b
    except ZeroDivisionError:
        return "Cannot divide by zero"
```

```
print(divide(10, 2))
print(divide(10, 0))
```

Output

```
5.0
Cannot divide by zero
```

Example 2: Validating User Input

```
try:
    age = int(input("Enter your age: "))
    if age < 0:
        raise ValueError("Age cannot be negative")
    print("Valid age:", age)
except ValueError as e:
    print("Error:", e)
```

Output

```
Enter your age: -3
Error: Age cannot be negative
```

Example 3: File Handling with finally

```
try:
    file = open("record.txt", "r")
    print(file.read())
except FileNotFoundError:
    print("The file does not exist.")
finally:
    print("File operation attempted.")
```

Output

```
The file does not exist.
File operation attempted.
```

Common Errors and Troubleshooting

A common mistake is using a bare `except:` clause, which catches all errors, including unrelated bugs, making debugging difficult.

Another frequent error is placing too much code inside the `try` block. This makes it hard to identify exactly where the error occurred.

Students often forget that the `else` block only runs when no exception is raised in the `try` block, not simply after the `except` block.

Another issue occurs when the `finally` block is expected to prevent an exception from propagating. The `finally` block always runs, but it does not stop the exception unless explicitly handled.

Raising exceptions without meaningful messages makes debugging harder, since the cause of the error is not clear to anyone reading the output.

Programming Exercises

Beginner Level

1. Write a program that catches a `ZeroDivisionError`.
2. Write a program that catches a `ValueError` when converting invalid input to an integer.
3. Write a program using `try` and `except` to handle an undefined variable (`NameError`).
4. Write a program that prints a custom message using the `finally block`.
5. Write a program that raises a `ValueError` if a number is negative.

Intermediate Level

1. Write a program that handles both `ZeroDivisionError` and `ValueError` in the same block.
2. Write a program that uses `else` to display a success message after a valid calculation.
3. Write a program that opens a file and handles `FileNotFoundError`.
4. Write a program that validates a password length and raises an exception if it is too short.
5. Write a program that catches an unknown exception using the base `Exception` class.

Advanced Level

1. Create a custom function that validates user age and raises a `ValueError` for invalid values.

2. Build a calculator program that handles division, invalid input, and unexpected errors.
3. Write a program that reads numbers from a list and skips invalid entries using exception handling.
4. Create a program that uses `try-except-else-finally` together in one structure.
5. Write a program that logs caught exceptions to a text file instead of printing them.

Lab Activities

Activity 1: Safe Division Calculator

Create a program that accepts two numbers and safely divides them.

Expected Steps:

1. Take two numbers as input.
2. `try` to attempt the division.
3. Catch `ZeroDivisionError` and `ValueError`.
4. Use `else` to display the result if successful.
5. Use `finally` to display a completion message.

Activity 2: Input Validator

Create a function that validates whether user input is a positive integer and raises a `ValueError` for invalid input.

Activity 3: File Reader with Error Handling

Create a program that attempts to open and read a file, handling `FileNotFoundError` and displaying an appropriate message if the file does not exist.

Assessment

Part A: Theory

1. Differentiate between a syntax error and a runtime exception.
2. Explain the purpose of the `finally` block.
3. What is the difference between `except ExceptionType` and a bare `except:`?
4. Explain the purpose of the `raise` keyword.
5. Why is it important to catch specific exceptions rather than using the base `Exception` class everywhere?

Part B: Programming

1. Write a program that handles a `ZeroDivisionError`.
2. Write a program that raises a `ValueError` for negative numbers.
3. Write a program using `try-except-else-finally` for a division operation.
4. Write a program that catches multiple exceptions using a grouped `except` clause.
5. Write a program that reads a file safely using exception handling.

Conclusion

Error handling is a fundamental component of writing reliable Python programs. The `try`, `except`, `else`, `finally`, and `raise` keywords together provide a comprehensive framework for anticipating, catching, and responding to runtime failures without crashing an application. Mastering exception handling prepares students for building resilient software for real-world applications such as data validation, file processing, web development, and automation, where uninterrupted and predictable program behavior is essential.